

# 딥러닝 언어처리

# 언어 처리

## ■ 개요

- 컴퓨터를 이용해 인간의 말을 알아듣는 연구는 딥러닝이 나오기 이전부터 계속 되어 왔음
- 하지만 언어의 규칙은 컴퓨터의 규칙과 달리 쉽게 해결되지 않는 여러 문제를 안고 있음
- 딥러닝이 등장하면서 자연어 처리 연구가 활발해지기 시작
- 이는 대용량 데이터를 학습할 수 있는 딥러닝의 속성 때문
- 즉, 비교적 쉽게 얻을 수 있는 자연어 데이터를 지속적으로 입력해 끊임없이 학습하는 것이 가능해졌기 때문

# 언어 처리

## ■ 개요

- 텍스트 자료를 모았다고 해서 이를 딥러닝에 그대로 입력할 수 있는 것은 아님
- 컴퓨터 알고리즘은 수치로 된 데이터만 이해할 뿐 텍스트는 이해할 수 없기 때문
- 따라서 텍스트를 정제하는 전처리 과정이 꼭 필요

# 텍스트의 토큰화

## ■ 토큰과 토큰화

- 텍스트를 잘게 나누는 것
- 입력할 텍스트가 준비되면 이를 단어별, 문장별, 형태소별로 나눌 수 있음
- 이렇게 작게 나누어진 하나의 단위를 **토큰(token)**이라함
- 입력된 텍스트를 잘게 나누는 과정을 **토큰화(tokenization)**라고 함
- 케라스가 제공하는 text 모듈의 **text\_to\_word\_sequence()** 함수를 사용하면 문장을 단어 단위로 쉽게 나눌 수 있음

# 텍스트의 토큰화

## ■ Text 모듈

```
from tensorflow.keras.preprocessing.text import text_to_word_sequence
```

```
# 전처리할 텍스트를 정합니다.
```

```
text = '해보지 않으면 해낼 수 없다'
```

```
# 해당 텍스트를 토큰화합니다.
```

```
result = text_to_word_sequence(text)
```

```
print("\n원문:\n", text)
```

```
print("\n토큰화:\n", result)
```

원문:

해보지 않으면 해낼 수 없다

토큰화:

['해보지', '않으면', '해낼', '수', '없다']

# 텍스트의 토큰화

## ■ 단어의 빈도수

- 각 단어가 몇 번이나 중복해서 쓰였는지 알 수 있음
- 단어의 빈도수를 알면 텍스트에서 중요한 역할을 하는 단어를 파악
- Bag-of-Words라는 방법이 이러한 전처리를 일컫는 말
- ‘단어의 가방(bag of words)’이라는 뜻 그대로, 같은 단어끼리 따로따로 가방에 담은 후 각 가방에 몇 개의 단어가 들어 있는지 세는 기법

# 텍스트의 토큰화

## ■ 단어의 빈도수

- 세 문장에 등장하는 단어를 모두 세어 보고 출현 빈도가 가장 높은 것부터 나열해 보면, '토큰화'가 3회, '딥러닝에서'가 2회, '텍스트의'가 2회, 그리고 나머지 단어들이 1회씩
- 가장 많이 사용된 단어인 '토큰화, 딥러닝, 텍스트'가 앞의 세 문장에서 중요한 역할을 하는 단어임

먼저 텍스트의 각 단어를 나누어 토큰화합니다.  
텍스트의 단어로 토큰화해야 딥러닝에서 인식됩니다.  
토큰화한 결과는 딥러닝에서 사용할 수 있습니다.

# 텍스트의 토큰화

## ■ 단어의 빈도수

- Tokenizer() 함수를 사용하면 단어의 빈도수를 쉽게 계산할 수 있음
- 케라스에서 제공하 옴 텍스트 전처리 함수 중 Tokenizer() 함수를 불러 옴

```
from tensorflow.keras.preprocessing.text import Tokenizer
```

- 전처리하려는 세 개의 문장을 docs라는 배열에 지정

```
docs = ['먼저 텍스트의 각 단어를 나누어 토큰화합니다.',  
        '텍스트의 단어로 토큰화해야 딥러닝에서 인식됩니다.',  
        '토큰화한 결과는 딥러닝에서 사용할 수 있습니다.',  
        ]
```



# 텍스트의 토큰화

## ■ 단어의 빈도수

- `Tokenizer()` 함수를 사용하면 단어의 빈도수를 쉽게 계산할 수 있음
- 토큰화 함수인 `Tokenizer()`를 이용해 전처리

```
token = Tokenizer()    # 토큰화 함수 지정
```

```
token.fit_on_texts(docs) # 토큰화 함수에 문장 적용
```

```
print("\n단어 카운트:\n", token.word_counts) # 단어의 빈도수를 계산한 결과 출력
```

- `word_counts`는 단어의 빈도수를 계산해 주는 함수

단어 카운트:

```
OrderedDict([('먼저', 1), ('텍스트의', 2), ('각', 1), ('단어를', 1), ('나누어', 1), ('토큰화', 1), ('합니다', 1), ('단어로', 1), ('토큰화해야', 1), ('딥러닝에서', 2), ('인식됩니다', 1), ('토큰화한', 1), ('결과는', 1), ('사용할', 1), ('수', 1), ('있습니다', 1)])
```

# 텍스트의 토큰화

## ■ 단어의 빈도수

- '토큰화'가 3회, '텍스트의'와 '딥러닝에서'가 2회, 나머지가 1회씩 나오고 있음
- 순서를 기억하는 OrderedDict 클래스에 담겨 있는 형태로 출력되는 것
- `document_count()` 함수를 이용하면 총 몇 개의 문장이 들어 있는지도 셀 수 있음

```
print("\n문장 카운트: ", token.document_count)
```

```
문장 카운트: 3
```

# 텍스트의 토큰화

## ■ 단어의 빈도수

- `word_docs()` 함수를 통해 각 단어들이 몇 개의 문장에 나오는지 세어서 출력
- 출력되는 순서는 랜덤

```
print("\n각 단어가 몇 개의 문장에 포함되어 있는가:\n", token.word_docs)
```

각 단어가 몇 개의 문장에 포함되어 있는가:

```
defaultdict(<class 'int'>, {'텍스트의': 2, '단어를': 1, '합니다': 1, '토큰화': 1, '먼저': 1, '각': 1, '나누어': 1, '인식됩니다': 1, '딥러닝에서': 2, '토큰화해야': 1, '단어로': 1, '수': 1, '사용할': 1, '결과는': 1, '있습니다': 1, '토큰화한': 1})
```

# 텍스트의 토큰화

## ■ 단어의 빈도수

- 각 단어에 매겨진 인덱스 값을 출력하려면 `word_index()` 함수를 사용

```
print("\n각 단어에 매겨진 인덱스 값:\n", token.word_index)
```

각 단어에 매겨진 인덱스 값:

```
{'텍스트의': 1, '딥러닝에서': 2, '먼저': 3, '각': 4, '단어를': 5, '나누어': 6, '토큰화': 7, '합니다': 8, '단어로': 9, '토큰화해야': 10, '인식됩니다': 11, '토큰화한': 12, '결과는': 13, '사용할': 14, '수': 15, '있습니다': 16}
```

## 단어의 원-핫 인코딩

## ■ 원-핫 인코딩(one-hot encoding)

- 단어가 문장의 다른 요소와 어떤 관계를 가지고 있는지 알아보는 방법이 필요
- 기법 중에서 가장 기본적인 방법인 원-핫 인코딩(one-hot encoding)
- 단어의 배열에 적용해 보는 것

'오랫동안 꿈꾸는 이는 그 꿈을 닮아간다'

- 각 단어를 모두 0으로 바꾸어 주고 원하는 단어만 1로 바꾸어 주는 것이 원-핫 인코딩

(0인덱스) 오랫동안 꿈꾸는 이는 그 꿈을 닮아간다

|     |   |   |   |   |   |     |
|-----|---|---|---|---|---|-----|
|     |   |   |   |   |   |     |
| [ 0 | 0 | 0 | 0 | 0 | 0 | 0 ] |

# 단어의 원-핫 인코딩

## ■ 원-핫 인코딩(one-hot encoding)

- 각 단어가 배열 내에서 해당하는 위치를 1로 바꾸어서 벡터화

|      |   |                   |
|------|---|-------------------|
| 오랫동안 | = | [ 0 1 0 0 0 0 0 ] |
| 꿈꾸는  | = | [ 0 0 1 0 0 0 0 ] |
| 이는   | = | [ 0 0 0 1 0 0 0 ] |
| 그    | = | [ 0 0 0 0 1 0 0 ] |
| 꿈을   | = | [ 0 0 0 0 0 1 0 ] |
| 달아간다 | = | [ 0 0 0 0 0 0 1 ] |

# 단어의 원-핫 인코딩

## ■ 원-핫 인코딩(one-hot encoding)

- 먼저 토큰화 함수를 불러와 단어 단위로 토큰화하고 각 단어의 인덱스 값을 출력

```
text = "오랫동안 꿈꾸는 이는 그 꿈을 닮아간다"

token = Tokenizer()
token.fit_on_texts([text])
print(token.word_index)
```

```
{'오랫동안': 1, '꿈꾸는': 2, '이는': 3, '그': 4, '꿈을': 5, '닮아간다': 6}
```

# 단어의 원-핫 인코딩

## ■ 원-핫 인코딩(one-hot encoding)

- 각 단어를 원-핫 인코딩 방식으로 표현
- Tokenizer의 `texts_to_sequences()` 함수를 사용해서 앞서 만들어진 토큰의 인덱스로만 채워진 새로운 배열

```
x = token.texts_to_sequences([text])
```

```
print(x)
```

```
[[1, 2, 3, 4, 5, 6]]
```



# 단어의 원-핫 인코딩

## ■ 원-핫 인코딩(one-hot encoding)

- 정수로 인덱스되어 있는 것을 0과 1로만 이루어진 배열로 바꾸어 주는 `to_categorical()` 함수를 사용해 원-핫 인코딩
- 배열 맨 앞에 0이 추가되므로 단어 수보다 1이 더 많게 인덱스 숫자를 잡아 주는 것에 유의

```
from tensorflow.keras.utils import to_categorical
```

```
# 인덱스 수에 하나를 추가해서 원-핫 인코딩 배열 만들기
```

```
word_size = len(token.word_index) + 1
```

```
x = to_categorical(x, num_classes=word_size)
```

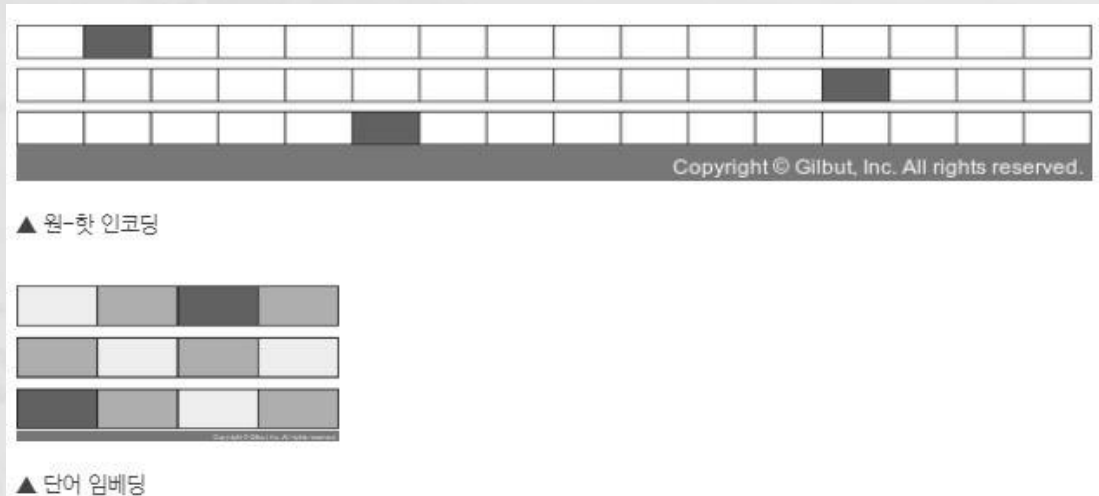
```
print(x)
```

|                          |      |
|--------------------------|------|
| [[[0. 1. 0. 0. 0. 0. 0.] | 오랫동안 |
| [0. 0. 1. 0. 0. 0. 0.]   | 꿈꾸는  |
| [0. 0. 0. 1. 0. 0. 0.]   | 이는   |
| [0. 0. 0. 0. 1. 0. 0.]   | 그    |
| [0. 0. 0. 0. 0. 1. 0.]   | 꿈을   |
| [0. 0. 0. 0. 0. 0. 1.]]  | 달아간다 |

# 단어 임베딩

## ■ 단어 임베딩

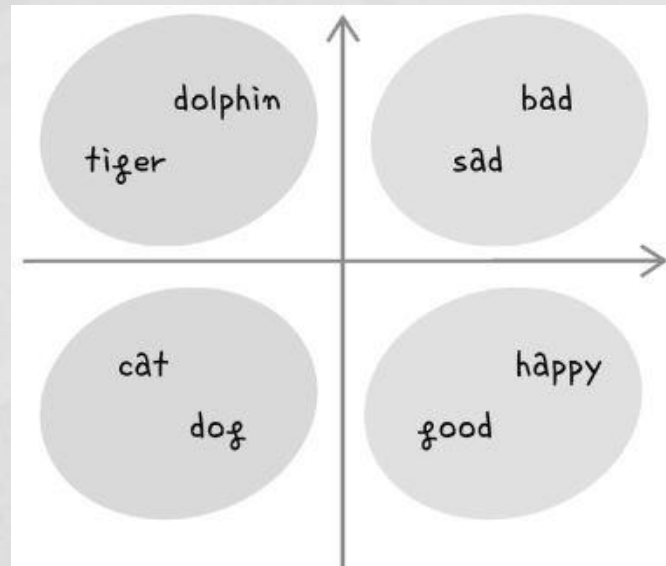
- 원-핫 인코딩을 그대로 사용하면 벡터의 길이가 너무 길어진다는 단점
- 공간적 낭비를 해결하기 위해 등장한 것이 단어 임베딩(word embedding)이라는 방법
- 단어 임베딩은 주어진 배열을 정해진 길이로 압축시킴
- 원-핫 인코딩을 사용해 만든 16차원 벡터가 단어 임베딩을 통해 4차원 벡터로 바뀐 예를 보기 쉽게 비교한 것



# 단어 임베딩

## ■ 단어 임베딩

- 단어 임베딩으로 얻은 결과가 밀집된 정보를 가지고 있고 공간의 낭비가 적다는 것
- 이러한 결과가 가능한 이유는 각 단어 간의 유사도를 계산했기 때문
- happy라는 단어는 bad보다 good에 더 가깝고, cat이라는 단어는 good보다는 dog에 가깝다는 것을 고려해 각 배열을 새로운 수치로 바꾸어 주는 것



# 단어 임베딩

## ■ 단어 임베딩

- 이 단어 간 유사도를 계산하는 것
- 적절한 크기로 배열을 바꾸어 주기 위해 최적의 유사도를 계산하는 학습 과정을 거치는 것
- Embedding() 함수를 사용
- Embedding() 함수는 입력과 출력의 크기를 나타내는 두 개의 매개변수
- Embedding(16,4)는 입력될 총 단어 수는 16, 임베딩 후 출력되는 벡터 크기는 4

```
from tensorflow.keras.layers import Embedding  
  
model = Sequential()  
model.add(Embedding(16,4))
```

- 단어를 매번 얼마나 입력할지 추가로 지정 : Embedding(16, 4, input\_length=2)

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 긍정, 부정 예측

- 영화를 보고 남긴 리뷰를 딥러닝 모델로 학습해서 각 리뷰가 긍정적인지 부정적인지를 예측하는 것
- 짧은 리뷰 열 개를 불러와 각각 긍정이면 1이라는 클래스를, 부정적이면 0이라는 클래스로 지정

```
# 텍스트 리뷰 자료를 지정합니다.
```

```
docs = ['너무 재밌네요', '최고예요', '참 잘 만든 영화예요', '추천하고 싶은 영화입니다.', '한 번 더 보고싶네요', '글쎄요', '별로예요', '생각보다 지루하네요', '연기가 어색해요', '재미없어요']
```

```
# 긍정 리뷰는 1, 부정 리뷰는 0으로 클래스를 지정합니다.
```

```
class = array([1,1,1,1,1,0,0,0,0,0])
```

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 긍정, 부정 예측

- Tokenize() 함수의 fit\_on\_texts를 이용해 각 단어를 하나의 토큰으로 변환

```
# 토큰화
token = Tokenizer()
token.fit_on_texts(docs)
print(token.word_index) # 토큰화된 결과를 출력해 확인합니다.
```

```
{'너무': 1, '재밌네요': 2, '최고예요': 3, '참': 4, '잘': 5, '만든': 6, '영화예요': 7, '추천하  
고': 8, '싶은': 9, '영화입니다': 10, '한번': 11, '더': 12, '보고싶네요': 13, '글쎄요': 14, '별  
로예요': 15, '생각보다': 16, '지루하네요': 17, '연기가': 18, '어색해요': 19, '재미없어요': 20}
```

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 긍정, 부정 예측

- 토큰에 지정된 인덱스로 새로운 배열을 생성

```
x = token.texts_to_sequences(docs)
print("\n리뷰 텍스트, 토큰화 결과:\n", x)
```

- 각 단어가 1부터 20까지의 숫자로 토큰화되었다는 것

리뷰 텍스트, 토큰화 결과:

```
[[1, 2], [3], [4, 5, 6, 7], [8, 9, 10], [11, 12, 13], [14], [15], [16, 17], [18, 19], [20]]
```

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 긍정, 부정 예측

- 딥러닝 모델에 입력하려면 학습 데이터의 길이가 동일
- 토큰의 수를 똑같이 맞추어 주어야 함
- 길이를 똑같이 맞추어 주는 작업을 **패딩**(padding) 과정
- 패딩 작업을 위해 `pad_sequences()` 함수를 사용
- `pad_sequences()` 함수를 사용하면 원하는 길이보다 짧은 부분은 숫자 0을 넣어서 채워 주고, 긴 데이터는 잘라서 같은 길이로 맞춤



# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 긍정, 부정 예측

- 생성한 x 배열을 네 개의 길이로 맞추기 위해

```
padded_x = pad_sequences(x, 4) # 서로 다른 길이의 데이터를 4로 맞추기  
print("\n패딩 결과:\n", padded_x)
```

패딩 결과:

```
[[ 0  0  1  2]  
 [ 0  0  0  3]  
 [ 4  5  6  7]  
 [ 0  8  9 10]  
 [ 0 11 12 13]  
 [ 0  0  0 14]  
 [ 0  0  0 15]  
 [ 0  0 16 17]  
 [ 0  0 18 19]  
 [ 0  0  0 20]]
```

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 단어 임베딩

- 단어 임베딩을 포함해 딥러닝 모델을 만들고 결과를 출력
- 임베딩 함수에 필요한 세 가지 파라미터는 '입력, 출력, 단어 수'
- 총 몇 개의 단어 집합에서(입력), 몇 개의 임베딩 결과를 사용할 것인지(출력), 그리고 매번 입력될 단어 수는 몇 개로 할지(단어 수)를 정해야 하는 것
- 총 몇 개의 인덱스가 '입력'되어야 하는지 정함
- word\_size라는 변수를 만든 후 길이를 세는 len() 함수를 이용해 word\_index 값을 앞서 만든 변수에 대입
- 전체 단어의 맨 앞에 0이 먼저 나와야 하므로 총 단어 수에 1을 더하는 것

`word_size = len(token.word_index) + 1`

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 단어 임베딩

- 몇 개의 임베딩 결과를 사용할 것인지, 즉 '출력'을 정할 차례
- word\_size만큼 입력 값을 이용해 여덟 개의 임베딩 결과
- 8이라는 숫자는 임의로 정한 것
- 만들어진 여덟 개의 임베딩 결과는 내부에서 계산해 딥러닝의 층으로 활용
- 매번 입력될 '단어 수'를 정함
- 패딩 과정을 거쳐 네 개의 길이로 맞추어 주었으므로 네 개의 단어가 들어가게 설정하면 임베딩 과정

Embedding(word\_size, 8, input\_length=4)

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 단어 임베딩

- 몇 개의 임베딩 결과를 사용할 것인지, 즉 '출력'을 정할 차례
- word\_size만큼 입력 값을 이용해 여덟 개의 임베딩 결과
- 8이라는 숫자는 임의로 정한 것
- 만들어진 여덟 개의 임베딩 결과는 내부에서 계산해 딥러닝의 층으로 활용
- 매번 입력될 '단어 수'를 정함
- 패딩 과정을 거쳐 네 개의 길이로 맞추어 주었으므로 네 개의 단어가 들어가게 설정하면 임베딩 과정

Embedding(word\_size, 8, input\_length=4)

# 텍스트를 읽고 긍정, 부정 예측하기

## ■ 딥러닝 모델

- 단어 임베딩을 포함해 딥러닝 모델을 만들고 결과를 출력
- 최적화 함수로 adam()을 사용하고 오차 함수로는 binary\_crossentropy()를 사용
- 20번 반복하고 나서 정확도를 계산해 출력

```
# 단어 임베딩을 포함해 딥러닝 모델을 만들고 결과를 출력합니다.
model = Sequential()
model.add(Embedding(word_size, 8, input_length=4))
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.fit(padded_x, classes, epochs=20)
print("\n Accuracy: %.4f" % (model.evaluate(padded_x, classes)[1]))
```

# 시퀀스 배열로 다루는 순환 신경망(RNN)

# 순환 신경망(Recurrent Neural Network, RNN)

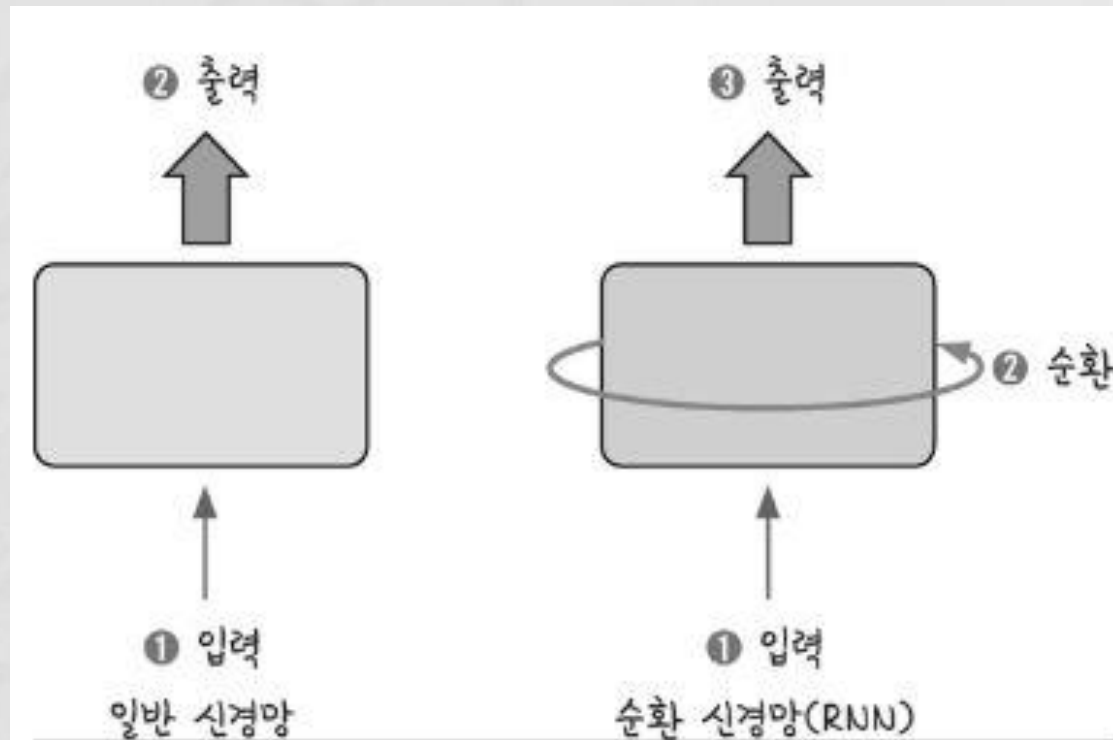
## ■ RNN

- 문장은 여러 개의 단어로 이루어져 있는데, 그 의미를 전달하려면 각 단어가 정해진 순서대로 입력되어야 하기 때문
- 여러 데이터가 순서와 관계없이 입력되던 것과는 다르게, 이번에는 과거에 입력된 데이터와 나중에 입력된 데이터 사이의 관계를 고려해야 하는 문제가 생기는 것
- 해결하기 위해 **순환 신경망(Recurrent Neural Network, RNN)** 방법이 고안
- 순환 신경망은 여러 개의 데이터가 순서대로 입력되었을 때 앞서 입력받은 데이터를 잠시 기억해 놓는 방법
- 기억된 데이터가 얼마나 중요한지 판단하고 별도의 가중치를 주어 다음 데이터로 넘어감

# 순환 신경망(Recurrent Neural Network, RNN)

## ■ RNN

- 모든 입력 값에 이 작업을 순서대로 실행하므로 다음 층으로 넘어가기 전에 같은 층을 맴도는 것처럼 보임
- 같은 층 안에서 맴도는 성질 때문에 순환 신경망(RNN)이라고 함

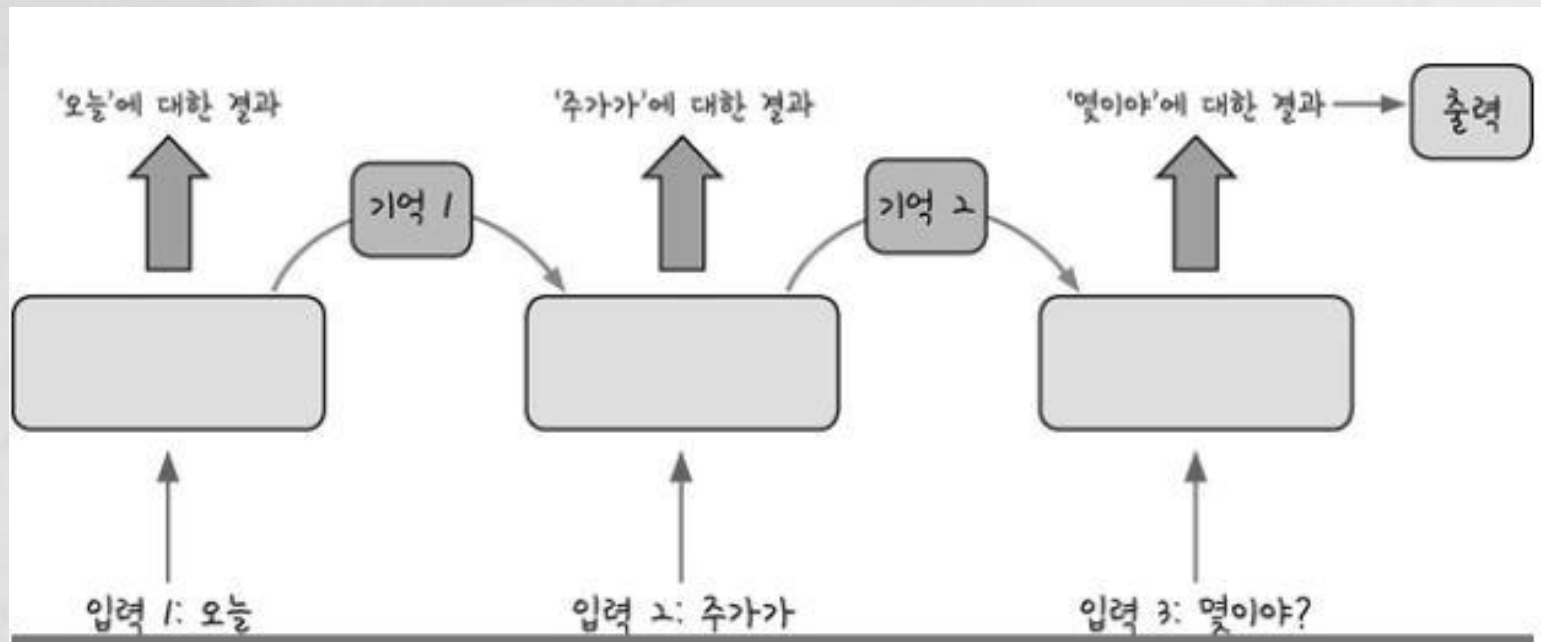




# 순환 신경망(Recurrent Neural Network, RNN)

## ■ RNN

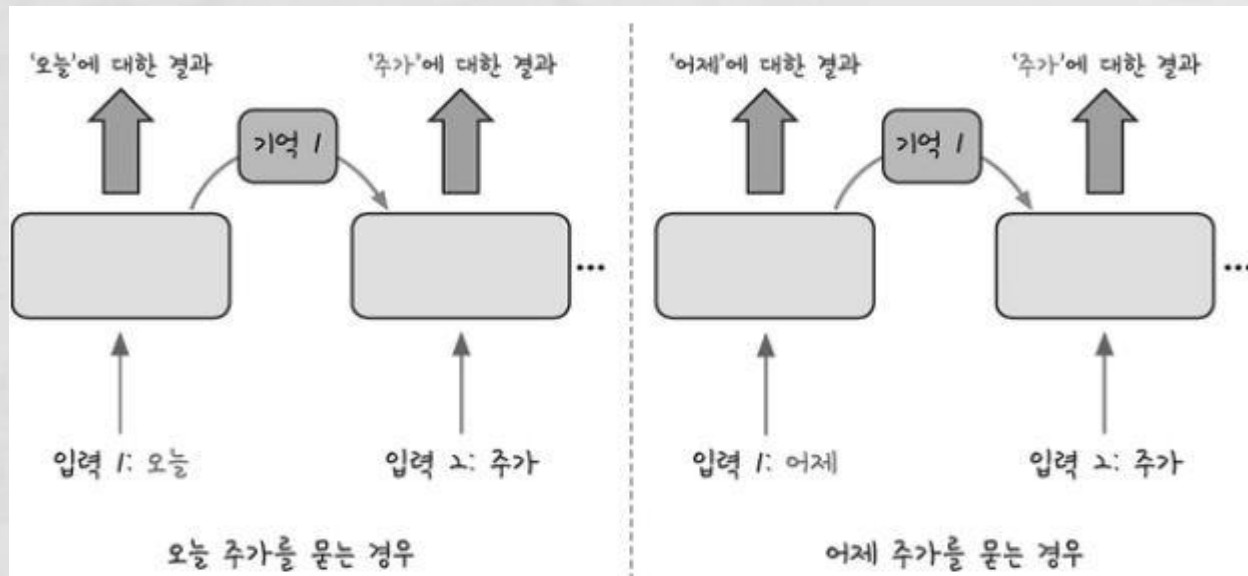
- “오늘 주가가 몇이야?”를 RNN이 처리하는 방식



# 순환 신경망(Recurrent Neural Network, RNN)

## ■ RNN

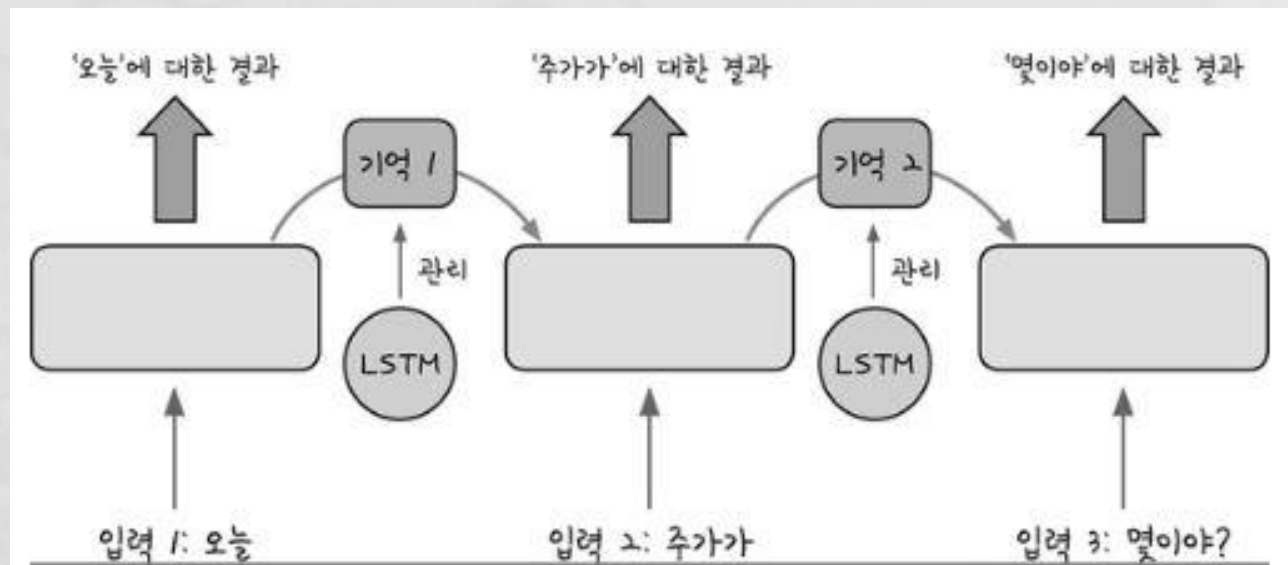
- 순환되는 중에 앞서 나온 입력에 대한 결과가 뒤에 나오는 입력 값에 영향을 주는 것
- 이렇게 해야 비슷한 두 문장이 입력되었을 때 그 차이를 구별해 출력 값에 반영
- 입력 2의 값은 양쪽 모두 '주가'이지만, 왼쪽은 '오늘'을 기준으로, 오른쪽은 '어제'를 기준으로 계산되어야 함



# LSTM(Long Short Term Memory)

## ■ LSTM(Long Short Term Memory) 방법

- LSTM은 한 층 안에서 반복을 많이 해야 하는 RNN의 특성상 일반 신경망보다 기울기 소실 문제가 더 많이 발생하고 이를 해결하기 어렵다는 단점을 보완한 방법
- 반복되기 직전에 다음 층으로 기억된 값을 넘길지 여부를 관리하는 단계를 하나 더 추가하는 것



# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- 입력된 문장 의미를 파악하는 것은 곧 모든 단어를 종합해 하나의 카테고리로 분류하는 작업이라고 할 수 있음
- 긴 텍스트를 읽고 이 데이터가 어떤 의미를 지니는지 카테고리로 분류

|                                    |       |
|------------------------------------|-------|
| 중부 지방은 대체로 맑겠으나, 남부 지방은 구름이 많겠습니다. | → 날씨  |
| 올 초부터 유동성의 힘으로 주가가 일정하게 상승했습니다.    | → 주식  |
| 이번 선거에서는 누가 이길 것 같아?               | → 정치  |
| 퍼셉트론의 한계를 극복한 신경망이 다시 뜨고 있대.       | → 딥러닝 |

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- 로이터 뉴스 데이터는 총 1만 1,228개의 뉴스 기사가 46개의 카테고리로 나뉘어진 대용량 텍스트 데이터

```
# 로이터 뉴스 데이터셋 불러오기  
from tensorflow.keras.datasets import reuters
```

- 불러온 데이터를 학습셋과 테스트셋으로 나눔

```
# 불러온 데이터를 학습셋과 데이터셋으로 나누기  
(X_train, y_train), (X_test, y_test) = reuters.load_data(num_words=10000, test_split=0.2)
```

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- 데이터를 확인한 후 출력

```
# 데이터를 확인한 후 출력해 보겠습니다.  
category = np.max(y_train) + 1  
print(category, '카테고리')  
print(len(X_train), '학습용 뉴스 기사')  
print(len(X_test), '테스트용 뉴스 기사')  
print(X_train[0])
```

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- `np.max()` 함수로 `y_train`의 종류를 구하니 46개의 카테고리로 구분되어 있음
- 이 중 8,982개는 학습용으로, 2,246개는 테스트용으로 준비
- 단어를 그대로 사용하지 않고 숫자로 변환한 후 학습할 수 있음
- 데이터 안에서 해당 단어가 몇 번이나 나타나는지 세어 빈도에 따라 번호를 붙였음(`tokenizer()` 같은 함수를 사용)

46 카테고리

8982 학습용 뉴스 기사

2246 테스트용 뉴스 기사

[1, 2, 2, 8, 43, 10, 447, 5, 25, 207...]

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- 모든 단어를 다 사용하는 것은 비효율적이므로 빈도가 높은 단어만 불러와 사용
- 이때 사용하는 인자가 바로 테스트셋과 학습셋으로 나눌 때 함께 적용했던 num\_words=1000의 의미
- 빈도가 1~1,000에 해당하는 단어만 선택해서 불러오는 것
- 각 기사의 단어 수가 제각각 다르므로 이를 동일하게 맞추어야 함
- 데이터 전처리 함수 sequence()를 이용
- maxlen=100은 단어 수를 100개로 맞추라는 의미

```
from tensorflow.keras.preprocessing import sequence

# 단어의 수를 맞추어 줍니다.
X_train = sequence.pad_sequences(X_train, maxlen=100)
X_test = sequence.pad_sequences(X_test, maxlen=100)
```



# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- y 데이터에 원-핫 인코딩 처리를 하여 데이터 전처리 과정

```
# 원-핫 인코딩 처리를 합니다.  
y_train = to_categorical(y_train)  
y_test = to_categorical(y_test)
```

- 딥러닝의 구조를 설정

```
# 모델의 구조를 설정합니다.  
model = Sequential()  
model.add(Embedding(1000, 100))  
model.add(LSTM(100, activation='tanh'))  
model.add(Dense(46, activation='softmax'))
```

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- Embedding 층과 LSTM 층을 새로 추가
- Embedding 층은 데이터 전처리 과정을 통해 입력된 값을 받아 다음 층이 알 수 있는 형태로 변환하는 역할
- Embedding('불러온 단어의 총수', '기사당 단어 수') 형식으로 사용하며, 모델 설정 부분의 맨 처음

```
# 모델의 구조를 설정합니다.  
model = Sequential()  
model.add(Embedding(10000, 100))  
model.add(LSTM(100, activation='tanh'))  
model.add(Dense(46, activation='softmax'))
```

# LSTM을 이용한 로이터 뉴스 카테고리 분류하기

## ■ 로이터 뉴스 카테고리 분류하기

- 모델 실행의 옵션을 정하고 ❷ 조기 중단 설정과 함께 ❸ 학습을 실행

```
# 모델의 실행 옵션을 정합니다.
```

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy']) -----
```

❶

```
# 학습의 조기 중단을 설정합니다.
```

```
early_stopping_callback = EarlyStopping(monitor='val_loss', patience=5) ----- ❷
```

```
# 모델을 실행합니다.
```

```
history = model.fit(X_train, y_train, batch_size=20, epochs=200, validation_data=(X_test, y_test), callbacks=[early_stopping_callback]) ----- ❸
```

# LSTM과 CNN의 조합을 이용한 영화 리뷰 분류하기

## ■ 영화 리뷰 분류하기

- 인터넷 영화 데이터베이스(Internet Movie DataBase, IMDB)는 영화와 관련된 정보와 출연진 정보, 개봉 정보, 영화 후기, 평점까지 매우 폭넓은 데이터가 저장된 자료
- 영화에 관해 남긴 2만 5,000여 개의 영화 리뷰가 담겨 있으며, 해당 영화를 긍정적으로 평가했는지 혹은 부정적으로 평가했는지도 담겨 있음

```
# 테스트셋을 지정합니다.  
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=5000)  
  
X_train = sequence.pad_sequences(x_train, maxlen=500)  
X_test = sequence.pad_sequences(x_test, maxlen=500)
```

# LSTM과 CNN의 조합을 이용한 영화 리뷰 분류하기

## ■ 영화 리뷰 분류하기

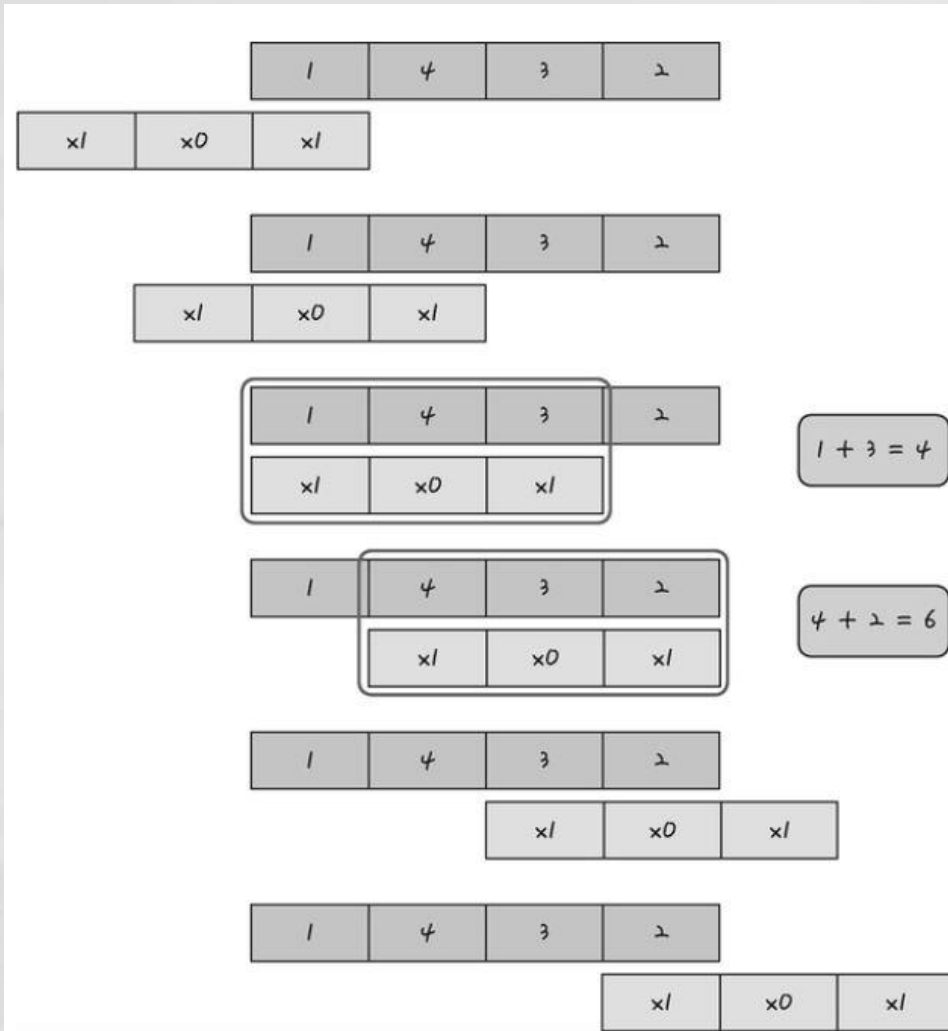
- 모델 구조를 설정
- `model.summary()` 함수를 이용해 현재 설정된 모델의 구조

```
# 모델의 구조를 설정합니다.  
model = Sequential()  
model.add(Embedding(5000, 100))  
model.add(Dropout(0.5))  
model.add(Conv1D(64, 5, padding='valid', activation='relu', strides=1))  
model.add(MaxPooling1D(pool_size=4))  
model.add(LSTM(55))  
model.add(Dense(1))  
model.add(Activation('sigmoid'))  
model.summary()
```

# LSTM과 CNN의 조합을 이용한 영화 리뷰 분류하기

## ■ 영화 리뷰 분류하기

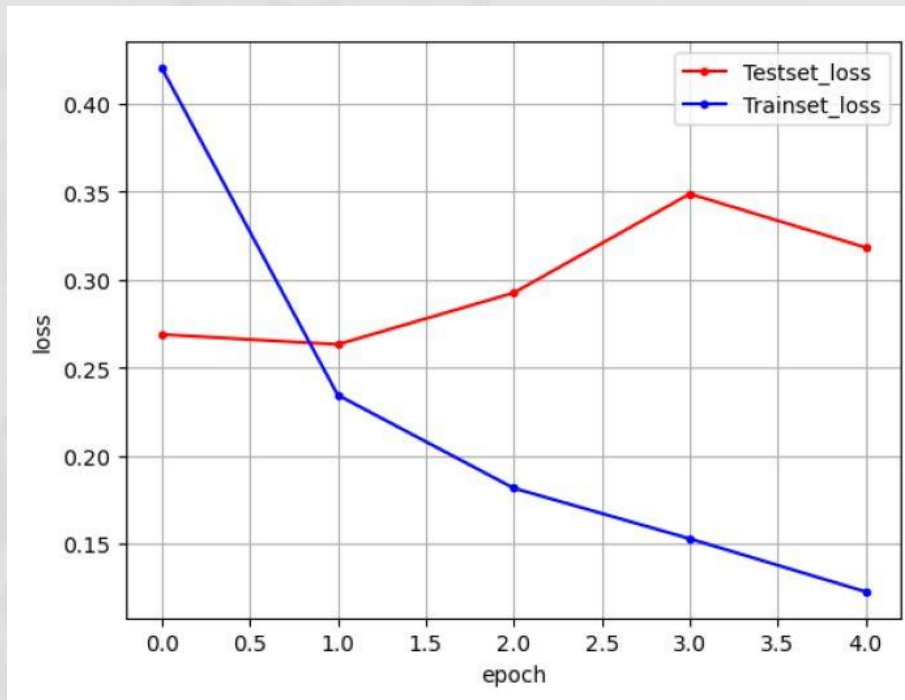
### ■ Conv1D의 개념



# LSTM과 CNN의 조합을 이용한 영화 리뷰 분류하기

## ■ 영화 리뷰 분류하기

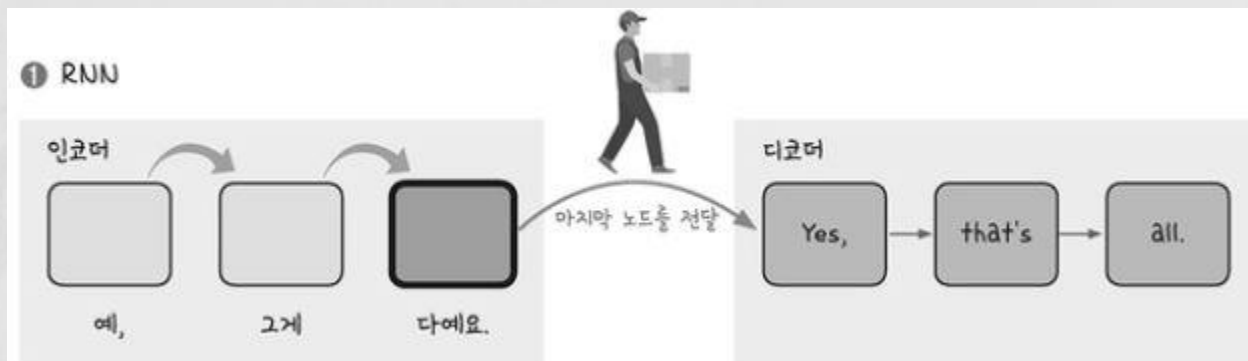
### ■ MaxPooling1D



# 어텐션(attention)을 사용한 신경망

## ■ RNN의 한계

- RNN은 여러 개의 입력 값이 있을 때 이를 바로 처리하는 것이 아니라 잠시 가지고 있는 것
- 입력된 값끼리 서로 관련이 있다면 이를 모두 받아 두어야 적절한 출력 값을 만들 수 있음
- 인코더에 입력된 각 셀 값을 하나씩 뒤로 보내다가, 맨 마지막 셀이 이 값을 디코더에 전달하는 것을 보여 줌
- 마지막 셀에 담긴 값에 전체 문장의 뜻이 함축되어 있으므로 이를 **문맥 벡터** (context vector)라고 함

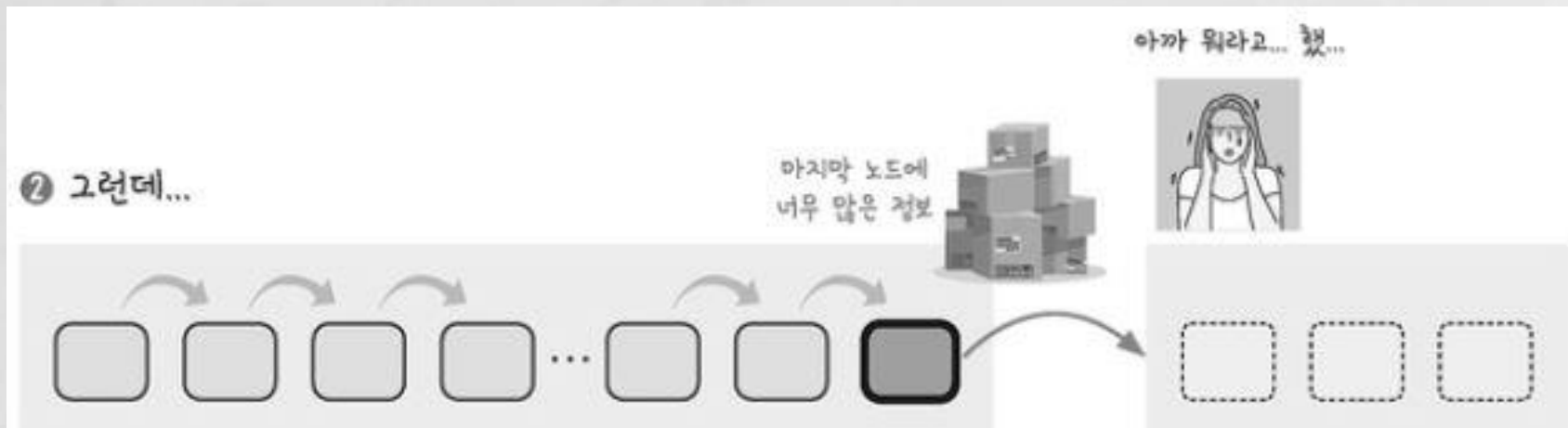




# 어텐션(attention)을 사용한 신경망

## ■ RNN의 한계

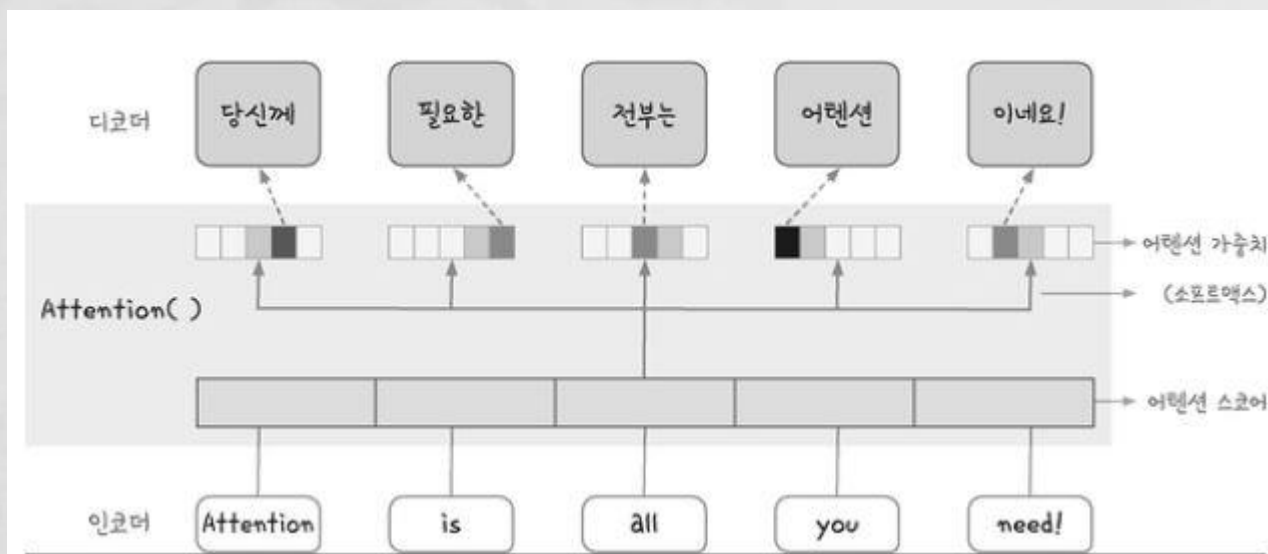
- 입력 값의 길이가 너무 길면 입력받은 셀의 결과들이 너무 많아진다는 것
- 입력이 길면 선두에서 전달받은 결과값이 중간에 희미해지기도 하고, 문맥 벡터가 모든 값을 제대로 디코더에 전달하기 힘들어지는 문제



# 어텐션(attention)을 사용한 신경망

## ■ 어텐션(attention) 알고리즘

- 먼저 인코더와 디코더 사이에 층이 하나 생김
- 새로 삽입된 층에는 각 셀로부터 계산된 스코어들이 모임
- 이 스코어를 이용해 소프트맥스 함수를 사용해서 어텐션 가중치를 만듦
- 이 가중치를 이용해 입력 값 중 어떤 셀을 중점적으로 볼지 결정



# 어텐션(attention)을 사용한 신경망

## ■ 어텐션(attention) 알고리즘

- 텐션을 실행하기 위해 어텐션(attention) 라이브러리가 필요

```
!pip install keras-self-attention
```

- 모델에 Attention() 함수가 들어갈 레이어

```
model.add(Attention())
```

```
# 모델의 구조를 설정합니다.
model = Sequential()
model.add(Embedding(5000, 500))
model.add(Dropout(0.5))
model.add(LSTM(64, return_sequences=True))
model.add(SeqSelfAttention(attention_activation="sigmoid"))
model.add(Dropout(0.5))
model.add(Flatten())
model.add(Dense(1))
model.add(Activation('sigmoid'))
```

# **GAN(Generative Adversarial Networks), 오토인코더**

# GAN(Generative Adversarial Networks), 오토인코더

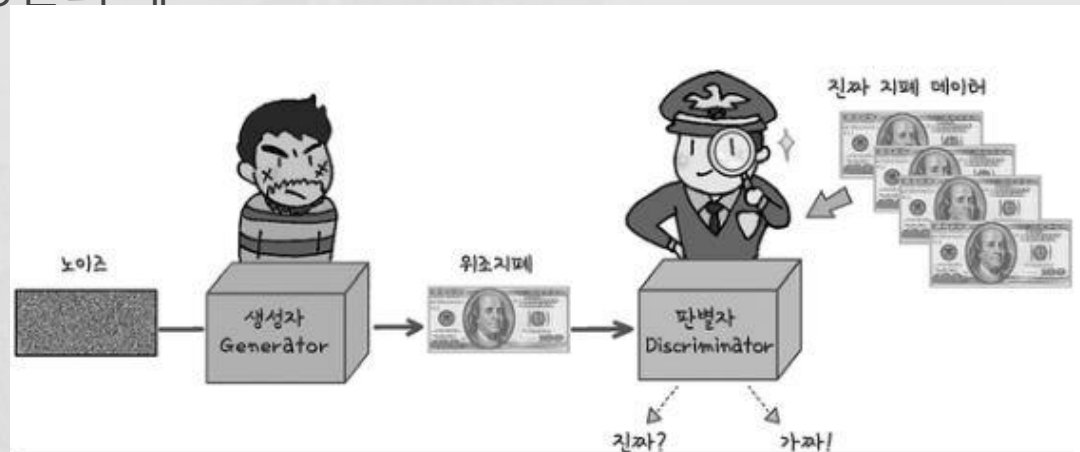
## ■ 얼굴 GAN, 오토인코더



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 얼굴 GAN, 오토인코더

- 생성적 적대 신경망(Generative Adversarial Networks), 줄여서 '**GAN**(간)'이라고 부르는 알고리즘을 이용해 만든 것
- GAN은 딥러닝의 원리를 활용해 가상 이미지를 생성하는 알고리즘
- GAN이라는 이름에는 적대적(adversarial, 서로 대립 관계에 있는)이란 단어가 들어 있는데, 이것은 GAN 알고리즘의 성격을 잘 말해 줌
- 진짜 같은 가짜를 만들기 위해 GAN 알고리즘 내부에서는 '적대적' 경합을 진행하기 때문
- GAN의 아이디어를 처음으로 제안한 이안 굿펠로(Ian Goodfellow)는 그의 논문에서 위조지폐범과 경찰의 예



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 얼굴 GAN, 오토인코더

- 가짜를 만들어 내는 파트를 '생성자(Generator)', 진위를 가려내는 파트를 '판별자(Discriminator)'라고 함
- 페이스북의 AI 연구 팀이 만들어 발표한 DCGAN(Deep Convolutional GAN)도 그중 하나
- DCGAN은 컨볼루션 신경망(CNN)을 GAN에 적용한 알고리즘

# GAN(Generative Adversarial Networks), 오토인코더

## ■ GAN 특징

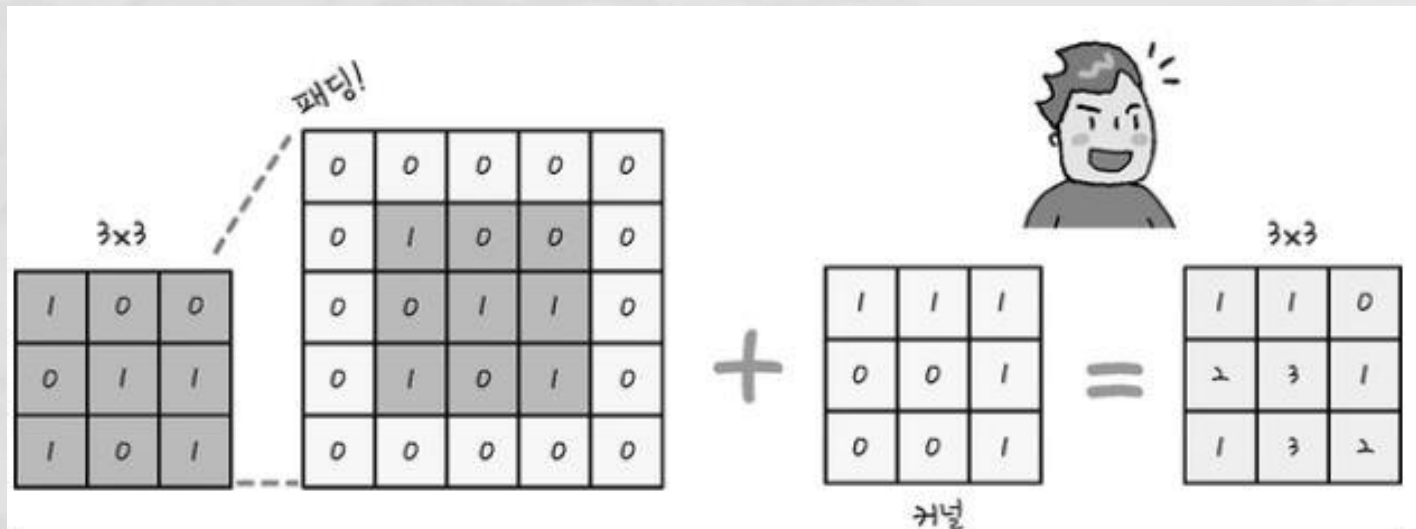
- 생성자(generator)는 가상의 이미지를 만들어 내는 공장
- 처음에는 랜덤한 픽셀 값으로 채워진 가짜 이미지로 시작해서 판별자의 판별 결과에 따라 지속적으로 업데이트하며 점차 원하는 이미지를 만들어 감
- DCGAN은 생성자가 가짜 이미지를 만들 때 컨볼루션 신경망(CNN)을 이용
- 옵티마이저(optimizer)를 사용하는 최적화 과정이나 컴파일하는 과정이 없다는 것
- 일부 매개변수를 삭제하는 풀링( pooling) 과정이 없는 대신 앞 장에서 배운 패딩( padding) 과정이 포함
- 빈 곳을 채워서 같은 크기로 맞추는 패딩 과정이 입력 크기와 출력 크기를 똑같이 맞추기 위해



# GAN(Generative Adversarial Networks), 오토인코더

## ■ GAN 특징

- 패딩 과정을 통해 생성자가 만들어 내는 이미지의 크기를 조절해야 하는 이유는 판별자가 비교할 '진짜'와 똑같은 크기가 되어야 하기 때문
- padding='same'이라는 설정을 통해 입력과 출력의 크기가 다를 경우 자동으로 크기를 확장해 주고, 확장된 공간에 0을 채워 넣을 수 있음



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 배치 정규화(Batch Normalization)

- 배치 정규화란 입력 데이터의 평균이 0, 분산이 1이 되도록 재배치하는 것
- 다음 층으로 입력될 값을 일정하게 재배치하는 역할
- BatchNormalization() 함수를 제공
- 생성자의 활성화 함수로는 ReLU() 함수를 쓰고 판별자로 넘겨주기 직전에는 tanh() 함수를 씀
- tanh() 함수를 쓰면 출력되는 값을 -1~1 사이로 맞출 수 있음

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 생성자 모델

- 7\*7입니다. 이는 이미지의 최초 크기를 의미
- UpSampling2D() 함수는 이미지의 가로세로 크기를 두 배씩 늘려 줌
- 작은 크기의 이미지를 점점 늘려 가면서 컨볼루션 층을 지나치게 하는 것이 DCGAN의 특징

```
generator = Sequential() # 모델 이름을 generator로 정하고 Sequential() 함수를 호출
generator.add(Dense(128*7*7, input_dim=100, activation=LeakyReLU(0.2))) ----- ❶
generator.add(BatchNormalization()) ----- ❷
generator.add(Reshape((7, 7, 128))) ----- ❸
generator.add(UpSampling2D()) ----- ❹
generator.add(Conv2D(64, kernel_size=5, padding='same')) ----- ❺
generator.add(BatchNormalization()) ----- ❻
generator.add(Activation(LeakyReLU(0.2))) ----- ❼
generator.add(UpSampling2D()) ----- ❽
generator.add(Conv2D(1, kernel_size=5, padding='same', activation='tanh')) ----- ❾
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 진위를 가려내는 장치, 판별자

- 가짜인지 진짜인지를 판별해 주는 장치인 **판별자**(discriminator)를 만들 차례
- 진짜(1) 아니면 가짜(0), 둘 중 하나를 결정하는 문제이므로 컴파일 부분 이진 로스 함수(binary\_crossentropy)와 최적화 함수(adam)
- 이 판별자는 가짜인지 진짜인지를 판별만 해 줄 뿐, 자기 자신이 학습을 해서는 안 된다는 것
- 판별자가 얻은 가중치는 판별자 자신이 학습하는 데 쓰이는 것이 아니라 생성자로 넘겨주어 생성자가 업데이트된 이미지를 만들도록 해야 함
- 판별자를 만들 때는 가중치를 저장하는 학습 기능을 꺼 주어야 함

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 진위를 가려내는 장치, 판별자

### ■ 판별자 모델

```
# 모델 이름을 discriminator로 정하고 Sequential() 함수를 호출합니다.  
discriminator = Sequential()  
discriminator.add(Conv2D(64, kernel_size=5, strides=2, input_shape=(28,28,1), padding="same")) ----- ❶  
discriminator.add(Activation(LeakyReLU(0.2))) ----- ❷  
discriminator.add(Dropout(0.3)) ----- ❸  
discriminator.add(Conv2D(128, kernel_size=5, strides=2, padding="same")) ----- ❹  
discriminator.add(Activation(LeakyReLU(0.2))) ----- ❺  
discriminator.add(Dropout(0.3)) ----- ❻  
discriminator.add(Flatten()) ----- ❼  
discriminator.add(Dense(1, activation='sigmoid')) ----- ❽  
discriminator.compile(loss='binary_crossentropy', optimizer='adam') ----- ❾  
discriminator.trainable = False ----- ❿
```

# GAN(Generative Adversarial Networks), 오토인코더

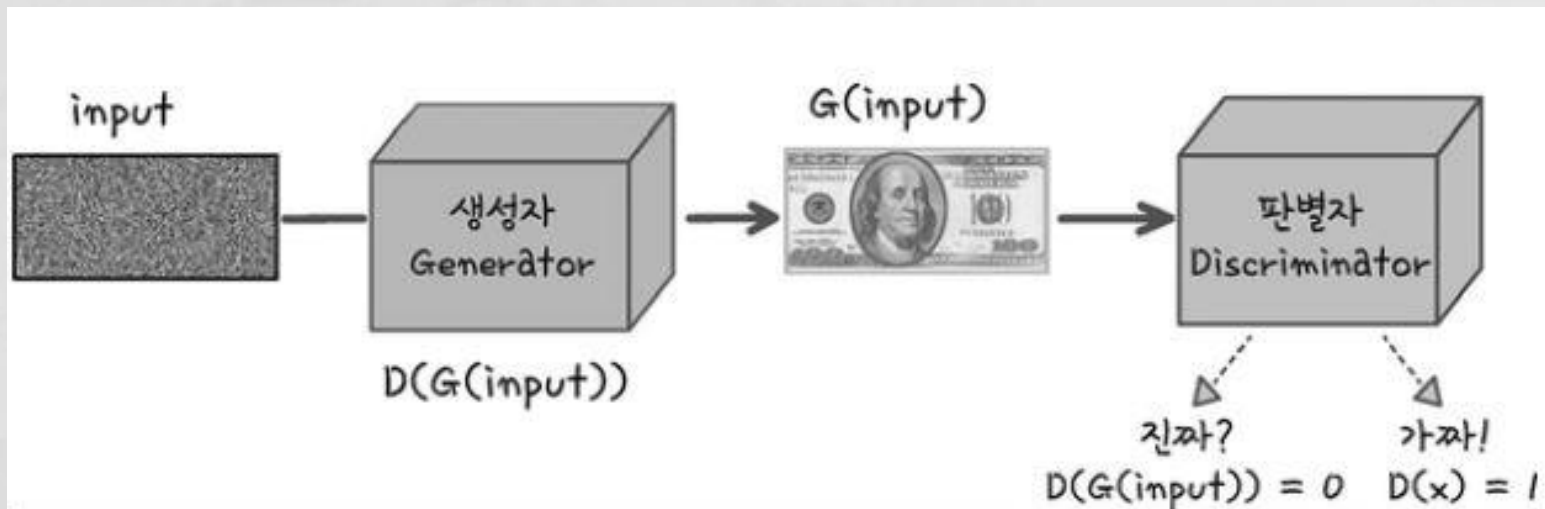
## ■ 진위를 가려내는 장치, 판별자

- `strides`는 커널 윈도우를 몇 칸씩 이동시킬지 정하는 옵션
- 특별한 설정이 없으면 커널 윈도우는 한 칸씩 움직
- `strides=2`라고 설정했다는 것은 커널 윈도우를 두 칸씩 움직이라는 뜻
- 판별의 결과가 진짜(1) 혹은 가짜(0), 둘 중에 하나가 되어야 하므로 `sigmoid()` 함수
- 판별이 끝나고 나면 판별자 자신이 학습되지 않게끔 학습 기능을 꺼 줌
- 이진 로스 함수(`binary_crossentropy`)와 최적화 함수(`adam`)를 써서 판별자에 필요한 준비를 마무리
- `discriminator.trainable = False`라는 옵션으로 이를 설정

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 적대적 신경망 실행하기

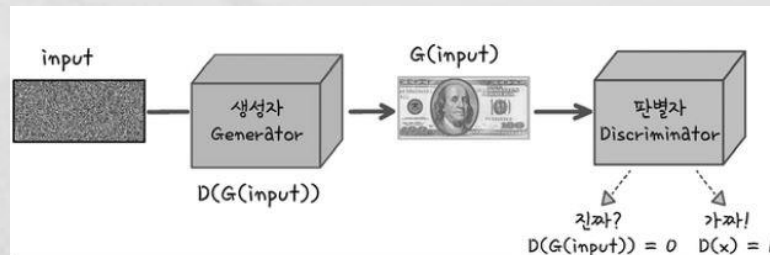
- 생성자와 판별자를 연결시키고 학습을 진행하며 기타 여러 가지 옵션을 설정하는 순서
- 생성자와 판별자를 연결시킨다는 것은 생성자에서 나온 출력을 판별자에 넣어서 진위 여부를 판별하게 만든다는 의미
- 생성자를  $G()$ , 판별자를  $D()$ , 실제 데이터를  $x$ , 입력 값을  $input$



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 적대적 신경망 실행하기

- 생성자  $G()$ 에 입력 값  $input$ 을 넣은 결과는  $G(input)$
- 판별자  $D()$ 에 넣은 결과는  $D(G(input))$ 이 됨
- 생성자는  $D(G(input))$ 이 참(1)이라고 주장하지만, 판별자는 실제 데이터인  $x$ 로 만든  $D(x)$ 만이 참이라고 여김
- 그러다 학습이 진행될수록 생성자가 만든  $G(input)$ 이 실제와 너무나 가까워져서 이것으로 만든  $D(G(input))$ 과 실제 데이터로 만든  $D(x)$ 를 잘 구별하지 못함
- 너무나 유사해진  $D(G(input))$ 과  $D(x)$ 를 판별자가 더는 구별하지 못하게 되어 정확도가 0.5에 가까워질 때, 비로소 생성자는 자신의 역할을 다하게 되어 학습은 종료





# GAN(Generative Adversarial Networks), 오토인코더

## ■ 적대적 신경망 실행하기

- 랜덤한 100개의 벡터를 케라스의 Input() 함수에 집어넣어 생성자에 입력할 ginput을 만드는 과정
- 판별자는 이 입력 값을 가지고 참과 거짓을 판별하는데, 그 결과를 dis\_output
- 케라스의 Model() 함수를 이용해 ginput 값과 ❷에서 구한 dis\_output 값을 넣어 gan이라는 이름의 새로운 모델
- 참과 거짓을 구분하는 이진 로스 함수(binary\_crossentropy)와 최적화 함수(adam)를 사용해 ❸에서 만든 gan 모델을 컴파일

```
ginput = Input(shape=(100,)) ----- ❶  
dis_output = discriminator(generator(ginput)) ----- ❷  
gan = Model(ginput, dis_output) ----- ❸  
gan.compile(loss='binary_crossentropy', optimizer='adam') ----- ❹
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 적대적 신경망 실행하기

- gan\_train() 함수를 사용해 학습이 진행
- poch, batch\_size 그리고 중간 과정을 저장할 때 몇 번마다 한 번씩 저장할지 정하는 saving\_interval

```
# 실행 함수를 선언합니다.  
def gan_train(epoch, batch_size, saving_interval): # 세 가지 변수 지정  
  
# MNIST 데이터 불러오기  
# MNIST 데이터를 다시 불러와 이용합니다. 단, 테스트 과정은 필요 없고  
# 이미지만 사용할 것이기 때문에 X_train만 호출합니다.  
(X_train, _), (_, _) = mnist.load_data()  
  
# 가로 28픽셀, 세로 28픽셀이고 흑백이므로 1을 설정합니다.  
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1).astype('float32')  
  
# 0~255 사이 픽셀 값에서 127.5를 뺀 후 127.5로 나누면 -1~1 사이 값으로 바뀝니다.  
X_train = (X_train - 127.5) / 127.5
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 오토인코더(Auto-Encoder, AE)

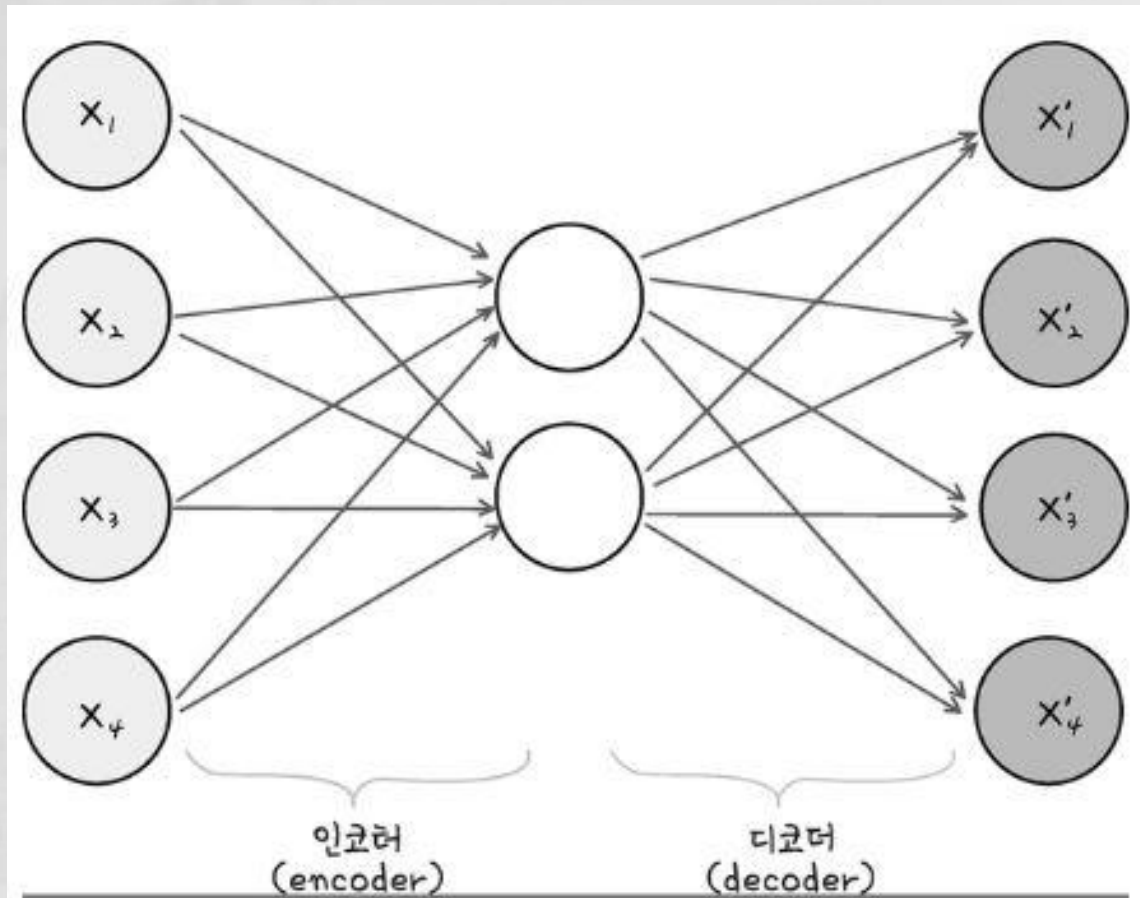
- GAN이 세상에 존재하지 않는 완전한 가상의 것을 만들어 내는 반면에, 오토인코더는 입력 데이터의 특징을 효율적으로 담아낸 이미지를 만들어짐
- GAN으로 사람의 얼굴을 만들면 진짜 같아 보여도 실제로는 존재하지 않는 완전한 가상 이미지가 만들어짐
- 오토인코더로 사람의 얼굴을 만들 경우 초점이 좀 흐릿하고 윤곽이 불명확하지만 사람의 특징을 유추할 수 있는 것들이 모여 이미지가 만들어짐



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 오토인코더(Auto-Encoder, AE)

- 오토인코더의 구조. 입력 값( $x$ )과 출력 값( $x'$ ) 사이에 차원이 작은 노드가 포함됨



# GAN(Generative Adversarial Networks), 오토인코더

## ■ 오토인코더(Auto-Encoder, AE)

### ▪ 인코딩과 디코딩 과정

```
# 생성자 모델 만들기
autoencoder = Sequential()

# 인코딩 부분
autoencoder.add(Conv2D(16, kernel_size=3, padding='same', input_shape=(28,
28,1), activation='relu')) ----- ❶
autoencoder.add(MaxPooling2D(pool_size=2, padding='same')) ----- ❷
autoencoder.add(Conv2D(8, kernel_size=3, activation='relu', padding='same')) ----- ❸
autoencoder.add(MaxPooling2D(pool_size=2, padding='same')) ----- ❹
autoencoder.add(Conv2D(8, kernel_size=3, strides=2, padding='same',
activation='relu')) ----- ❺
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 오토인코더(Auto-Encoder, AE)

### ▪ 인코딩과 디코딩 과정

```
# 디코딩 부분
autoencoder.add(Conv2D(8, kernel_size=3, padding='same',
activation='relu')) ----- ⑥
autoencoder.add(UpSampling2D()) ----- ⑦
autoencoder.add(Conv2D(8, kernel_size=3, padding='same',
activation='relu')) ----- ⑧
autoencoder.add(UpSampling2D()) ----- ⑨
autoencoder.add(Conv2D(16, kernel_size=3, activation='relu')) ----- ⑩
autoencoder.add(UpSampling2D()) ----- ⑪
autoencoder.add(Conv2D(1, kernel_size=3, padding='same', activation='sigmoid')) ----- ⑫

# 전체 구조 확인
autoencoder.summary() ----- ⑬
```



# GAN(Generative Adversarial Networks), 오토인코더

## ■ ImageDataGenerator() 함수

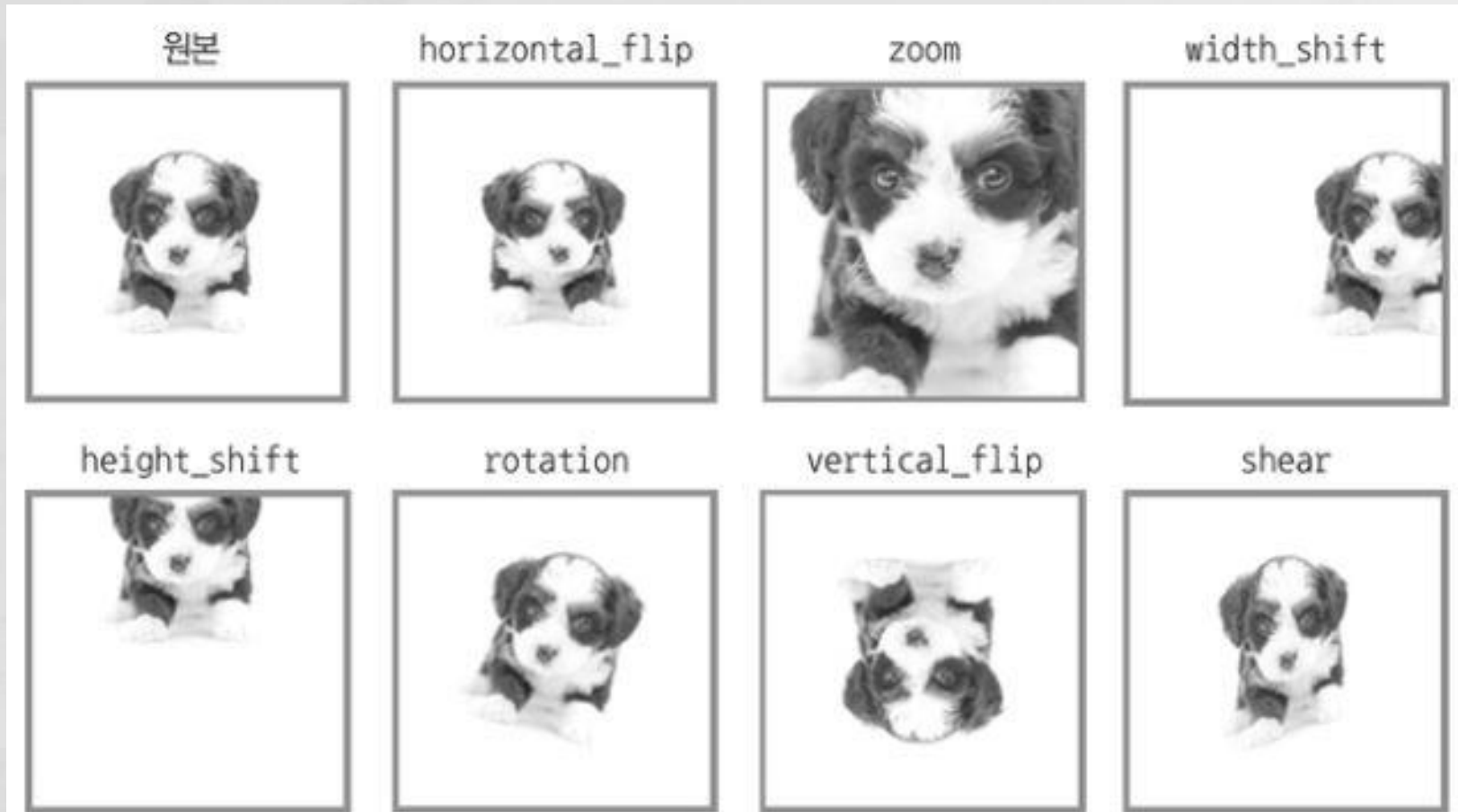
- rescale: 주어진 이미지의 크기를 바꾸어 줌 예를 들어 원본 영상이 0~255의 RGB 값을 가지고 있으므로 255로 나누면 0~1의 값으로 변환되어 학습이 좀 더 빠르고 쉬워짐
- horizontal\_flip, vertical\_flip: 주어진 이미지를 수평 또는 수직으로 뒤집어짐
- zoom\_range: 정해진 범위 안에서 축소 또는 확대
- width\_shift\_range, height\_shift\_range: 정해진 범위 안에서 그림을 수평 또는 수직으로 랜덤하게 평행 이동
- rotation\_range: 정해진 각도만큼 이미지를 회전
- shear\_range: 좌표 하나를 고정시키고 다른 몇 개의 좌표를 이동시키는 변환
- fill\_mode: 이미지를 축소 또는 회전하거나 이동할 때 생기는 빈 공간을 어떻게 채울지 결정. nearest 옵션을 선택하면 가장 비슷한 색으로 채워짐



# GAN(Generative Adversarial Networks), 오토인코더

## ■ ImageDataGenerator() 함수

- ImageDataGenerator 옵션의 결과



# GAN(Generative Adversarial Networks), 오토인코더

## ■ flow\_from\_directory() 함수

- 이미지 생성 옵션을 정하고 나면 실제 데이터가 있는 곳을 알려 주고 이미지를 불러오는 작업을 해야 함

```
train_generator = train_datagen.flow_from_directory(
    './data-ch20/train',  # 학습셋이 있는 폴더 위치
    target_size=(150,150), # 이미지 크기
    batch_size=5,
    class_mode='binary')  # 치매/정상 이진 분류이므로 바이너리 모드로 실행
```

```
test_generator = test_datagen.flow_from_directory(
    './data-ch20/test'    # 테스트셋이 있는 폴더 위치
    target_size=(150,150),
    batch_size=5,
    class_mode='binary')
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ flow\_from\_directory() 함수

- ## ■ 모델 실행을 위한 옵션

```
# 모델의 실행 옵션을 설정합니다.
model.compile(loss='binary_crossentropy', optimizer=optimizers.Adam
(learning_rate=0.0002), metrics=['accuracy']) -----❶

# 학습의 조기 중단을 설정합니다.
early_stopping_callback = EarlyStopping(monitor='val_loss', patience=5)

# 모델을 실행합니다.
history = model.fit(train_generator, -----❷
    epochs=100,
    validation_data=test_generator, -----❸
    validation_steps=10,
    callbacks=[early_stopping_callback])
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ 치매 환자의 뇌인지 일반인의 뇌인지 예측하기

### ■ 학습셋의 변형을 설정

```
# 학습셋의 변형을 설정하는 부분입니다.  
train_datagen = ImageDataGenerator(rescale=1./255, # 주어진 이미지 크기를 설정  
    horizontal_flip=True, # 수평 대칭 이미지를 50% 확률로 만들어 추가합니다.  
    width_shift_range=0.1, # 전체 크기의 15% 범위에서 좌우로 이동합니다.  
    height_shift_range=0.1, # 마찬가지로 위아래로 이동합니다.  
    #rotation_range=5, # 정해진 각도만큼 회전시킵니다.  
    #shear_range=0.7, # 좌표 하나를 고정시키고 나머지를 이동시킵니다.  
    #zoom_range=[0.9, 2.2], # 확대 또는 축소시킵니다.  
    #vertical_flip=True, # 수직 대칭 이미지를 만듭니다.  
    #fill_mode='nearest' # 빈 공간을 채우는 방법입니다.  
    # nearest 옵션은 가장 비슷한 색으로 채우게 됩니다.  
)
```

# GAN(Generative Adversarial Networks), 오토인코더

## ■ flow\_from\_directory() 함수

- 모델 실행을 위한 옵션

# 정리

---

## 정리

- 딥러닝 언어 처리
- RNN